

Indexes — Fast Lookups over the Ledger

Why a key-value store by hash isn't enough, and the derived tables that answer the questions wallets and the node actually ask — UTXO, address, tokens, height, mempool.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 28 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Indexes · Derived state · UTXO index · Address index · Height index · Mempool tips ·
IndexesManager · Backends · Rebuildable · Index sync

Table of Contents

Chapter 28 — Indexes: Fast Lookups over the Ledger	3
28.1 Localization	4
28.2 What it does and why it exists	4
The access-pattern problem	5
The classic answer: an index	5
The property that makes indexes safe: they are derived	6
28.3 The code, walked	7
28.3.1 A toy first: index a list of records by a field	7
28.3.2 BaseIndex: the shared interface	8
28.3.3 Scope: declaring what a vertex an index wants	9
28.3.4 IndexesManager: one object owns them all	10
28.3.5 Two backends, the same manager	11
28.3.6 The UTXO index — the payoff of Chapter 7	12
28.3.7 The other indexes, in brief	14
28.3.8 Critical vs. non-critical, and why the split	16
28.4 A worked trace: one transaction moves through the indexes	17
28.5 How indexes are rebuilt — and why they can be	17
28.6 How it plugs into the lifecycle	19
Recap	19

Chapter 28 — Indexes: Fast Lookups over the Ledger

What you'll learn in this chapter

- What an **index**¹ is, in the general database sense, and the access-pattern problem it solves: storage can fetch a vertex by its hash, but cannot answer "which outputs can this address spend?" or "what block is at height 800,000?" without reading the whole ledger.
- The trade-off every index makes: it spends **disk space and update work** at write time to buy **fast answers** at read time.
- Why an index holds **no truth of its own** — it is derived state, always recomputable from the vertices in storage, which is exactly what makes it safe to rebuild after a crash.
- The real indexes Hathor keeps — **UTXO, address, tokens, height, timestamp/sorted, mempool tips**, and the info/counts and nano-contract indexes — what each one maps and who asks for it.
- How the **IndexesManager** owns them all behind one object, offered in two interchangeable backends (in-memory and RocksDB), the same abstraction pattern you met at storage.
- How indexes are **kept in sync** as vertices are executed and voided, and how they are **rebuilt from scratch** at boot when their on-disk state is stale.

The previous chapter (27) gave the node a place to put vertices and get them back: a key-value store, keyed by each vertex's hash. That answers exactly one kind of question — "give me the vertex whose id is X" — and answers it fast. But almost nothing a real user wants to know is phrased that way. A wallet asks "what is my balance?"; a block explorer asks "what is the latest block?"; the sync protocol asks "what is the tip of the chain?"; the mempool asks "which transactions are not yet confirmed?" None of these is a lookup by hash, and none of them can be answered without machinery that the bare key-value store does not have. That machinery is `hathor/indexes/`. This chapter is about the derived lookup tables the node maintains on top of storage so those questions become cheap.

28.1 Localization

`hathor/indexes/` sits in the **storage-and-lookup** group of the module map (Chapter 0, §0.4), directly above the storage layer of Chapter 27 and directly below the manager (Chapter 29) and the read-facing APIs that consume it. It depends on the vertex model (Chapter 25) and on storage; almost nothing depends on it except the manager and the APIs.

```
hathor/
├── transaction/
│   └── storage/                ← KV-by-hash: fetch a vertex by its id (Ch 27)
└── indexes/                  ◀ YOU ARE HERE – derived lookup tables over storage
    ├── manager.py            ← IndexesManager (ABC) + RocksDBIndexesManager
    ├── base_index.py         ← BaseIndex (ABC): the shared index interface
    ├── scope.py              ← Scope: which vertices an index cares about (rebuild engine)
    └── — the real indexes (each = abstract interface + 2 backends) —
        ├── utxo_index.py     ← unspent outputs, keyed by (address, token)      → "what can I
        ├── address_index.py  ← address → its transaction history                → "my history?"
        ├── tokens_index.py   ← token uid → info + supply + holders              → "tell me abo
        ├── height_index.py   ← block height → block hash (+ the height tip)     → "the chain
        ├── timestamp_index.py ← vertices sorted by timestamp (all/blocks/txs)   → "give me a
        ├── mempool_tips_index.py ← the tips of the unconfirmed mempool          → "what's pen
        ├── info_index.py     ← global counts + first/latest timestamps          → node statis
        ├── nc_creation_index.py ← nano-contract creations
        ├── nc_history_index.py ← per-contract call history                    ] → Ch 39
        └── blueprint_*_index.py ← blueprint listing + history
    └── — per-index backends —
        ├── memory_*_index.py ← in-RAM implementations (tests, simulator)
        └── rocksdb_*_index.py ← persisted implementations (production)
```

Context. `hathor/indexes/` answers one question for the rest of the node: "given the ledger, look this up fast." Storage (Ch 27) can return a vertex by its hash and nothing else; this package precomputes every other way the node and its clients want to slice the ledger — by address, by token, by height, by time, by mempool membership. It is read constantly by the APIs (Ch 36/40), the wallet, the sync protocol, and mining, and it is kept current by consensus (Ch 32) and ingestion (Ch 33). Because every entry in it is derived from vertices that storage already holds, the whole package can be thrown away and rebuilt — and that is exactly what the node does at boot when an index is stale.

28.2 What it does and why it exists

Start with the problem in its most general form, before any Hathor code.

The access-pattern problem

A key-value store is a dictionary: you hand it a key, it hands you back the value. Hathor's storage is exactly that, with the key being a vertex's 32-byte hash and the value being the serialized vertex (Chapter 27). This is the right primitive — it is the one lookup that the data structure makes naturally, because a vertex is named by its hash.

But consider the questions a node is actually asked all day:

1. "How much HTR does address `H...abc` own?" — There is no field anywhere that stores a balance (Chapter 7: the UTXO model has no balances). The answer is the sum of the values of every unspent output locked to that address. To compute it from raw storage you would have to read **every transaction ever made**, check each output's script, and track which were later spent. For a ledger with millions of vertices, that is seconds-to-minutes of work for one balance query.
2. "What is the block at height 800,000?" — A block does not store "I am at height 800,000" in a way storage can search on; the hash tells you nothing about the height. From raw storage you would walk the block backbone from a known point counting blocks.
3. "Give me transactions between 9:00 and 9:05 this morning." — Storage has no notion of time-order. You would scan everything and filter.
4. "What transactions has address `H...abc` ever been part of?" — Again, a full scan.

The common shape: **the question keys on something other than the hash** (an address, a height, a timestamp, a token), and **storage is keyed only on the hash**. Answering by brute-force scan is correct but unusably slow.

The classic answer: an index

This is not a blockchain problem; it is a database problem, and databases have solved it for fifty years with **indexes**. The idea is the same as the index at the back of a book. The book's pages are stored in page order (the equivalent of "by hash" — the natural physical order). If you want to find every page that mentions "Pydantic," reading the whole book is the brute-force scan. Instead the publisher precomputes, once, a sorted list — `Pydantic → pages 18, 192, 269` — and prints it at the back. Finding the topic is now instant.

A database index is the same trade made in software: pick a question you will ask often, precompute a lookup table keyed the way that question is phrased, and keep that table updated as the data changes. You pay two ongoing costs — **extra space** (the table) and **extra work on every write** (you must update the table when the data

changes) — in exchange for turning a full scan into a direct lookup. The node's indexes are exactly this: an address-keyed table, a height-keyed table, a time-sorted table, and so on, each precomputing the answer to one family of questions.

The property that makes indexes safe: they are derived

Here is the conceptual point to hold above all others, because it explains the whole design of the package.

An index holds no truth of its own. Every entry in every index is computed from vertices that already live in storage. The height index says "height 800,000 → hash 0000ab..." — but that fact is already implied by the blocks in storage; the index merely caches it in a convenient shape. The UTXO index says "address A has these three unspent outputs" — but that, too, is fully determined by the transactions in storage. Nothing is ever recorded only in an index.

This makes an index **derived** (sometimes called secondary) state, as opposed to the **primary** state in storage. And derived state has a defining property: **it can always be recomputed from the primary state**. If an index is lost, corrupted, or never built, the node can reconstruct it by replaying the vertices in storage through the index's update logic — exactly as you could regenerate a book's back-index by re-reading the book.

Why does this matter so much? Two reasons:

- **Crash recovery.** If the node dies mid-write, an index on disk may be half-updated and therefore wrong. Because the index is derived, the safe response is not delicate repair but blunt rebuild: clear it and replay. The node does precisely this at boot (§28.5). The primary state in storage is the only thing that must survive a crash intact; the indexes are disposable.
- **It bounds the danger.** A bug in index code can give a wrong answer to a query, but it can never corrupt the ledger, because the ledger does not live in the index. The worst case is a stale lookup table, fixable by a rebuild. This is why some indexes are even allowed to be skipped entirely unless a feature needs them (§28.4).

Recap — storage / KV-by-hash (full treatment in Ch. 27). Hathor's persistence layer is a key-value store: each vertex is written under its 32-byte hash and read back by that hash. It is backed by **RocksDB** (an embedded on-disk key-value database). It is the **primary** store of the ledger — the source of truth. Its one fast operation is "fetch the vertex with this id." Everything in this chapter is built on top of it to support the other ways the node wants to look data up. → full treatment in Ch. 27.

Recap — the UTXO model (full treatment in Ch. 7). Money in Hathor is a set of discrete **unspent transaction outputs** (UTXOs). There is no stored balance and no account; an address's balance is the sum of the unspent outputs locked to it. A transaction consumes some outputs (its inputs) and creates new ones; an output that has been consumed is "spent" and no longer counts. Computing a balance therefore means finding the currently unspent outputs for an address — exactly the job of the UTXO index. → full treatment in Ch. 7.

Recap — DAG tips and the mempool (full treatment in Ch. 8 & Ch. 0). Hathor's ledger is a directed acyclic graph of vertices. The **tips** are the vertices nothing else confirms yet — the frontier of the graph. The **mempool** is the set of valid transactions the node knows about that have been seen but not yet confirmed by a block. The mempool tips are the tips that are still unconfirmed; tracking them lets the node hand out parents for new transactions and report what is pending. → Ch. 8 (DAG/tips), Ch. 0 (mempool).

Recap — interchangeable backends (full treatment in Ch. 1 & Ch. 5). When two implementations expose the same methods, code written against the shared interface works with either, and you can swap them at construction time. Hathor uses this for storage (memory vs. RocksDB) and again here: every index is an abstract interface with a `Memory...` and a `RocksDB...` implementation, and an `IndexesManager` that bundles one full set. The rest of the node never knows which it holds. This is the Liskov-substitutable interface idea from object orientation, applied to lookup tables. → Ch. 1 (ABCs/interfaces), Ch. 5 (typing).

28.3 The code, walked

28.3.1 A toy first: index a list of records by a field

Strip the idea to its bones. Suppose you have a list of payment records and you keep being asked "give me all payments from a given person." The records are stored in arrival order (the "by hash" of our toy). The naive answer scans the whole list every time:

```
records = [
    {"id": 1, "frm": "alice", "to": "bob", "amt": 30},
    {"id": 2, "frm": "alice", "to": "carol", "amt": 10},
    {"id": 3, "frm": "bob", "to": "alice", "amt": 5},
]

def payments_from(name):
    # 0(n) every call – the brute-force scan
    return [r for r in records if r["frm"] == name]
```

An **index** precomputes the answer once, keyed the way the question is asked — here, a dict from sender to the list of their record ids:

```
from collections import defaultdict

by_sender: dict[str, list[int]] = defaultdict(list)  # the index

def add_record(r):                                # update the index on every write...
    records.append(r)
    by_sender[r["frm"].append(r["id"])

def payments_from(name):                          # ...so the read is now O(1)
    return by_sender.get(name, [])
```

Three observations carry straight into Hathor:

- **The index costs space and write-work.** `by_sender` is extra memory, and `add_record` now does extra work on every insert. You paid at write time to make reads cheap.
- **The index is derived.** `by_sender` contains nothing that is not already in `records`. Delete it and you can rebuild it by replaying `records` through `add_record`. (Hold this — it is \$28.5.)
- **Removal must be handled too.** If a record can be retracted, you must also remove its id from `by_sender`, or the index will report stale answers. Every real index has an add path and a remove path — and in Hathor, an executed path and a voided path, which are add and remove under different names.

Hathor's indexes are this skeleton, scaled up: the keys are addresses, heights, timestamps and tokens; the values are vertex hashes; the backing store can be RAM or RocksDB; and "add/remove" become "a vertex was executed / a vertex was voided."

28.3.2 `BaseIndex` : the shared interface

Every index inherits from one abstract base, `BaseIndex` (`base_index.py:32`). Its own docstring states the purpose: "so we can interact with indexes without knowing anything specific to [their] implementation... to generalize how we initialize indexes and keep track of which ones are up-to-date." It is small:

```
# hathor/indexes/base_index.py:32
class BaseIndex(ABC):
    def __init__(self, *, settings: HathorSettings) -> None:
        self.settings = settings
        self.log = logger.new()

    def init_start(self, indexes_manager: 'IndexesManager') -> None:
        pass # optional hook; most indexes don't need it

    @abstractmethod
    def get_scope(self) -> Scope: ... # which vertices do I care about? (:49)

    @abstractmethod
    def get_db_name(self) -> Optional[str]: ... # my on-disk state key, or None (:54)

    @abstractmethod
    def init_loop_step(self, tx: BaseTransaction) -> None: ... # rebuild: feed me a tx (:63)

    @abstractmethod
    def force_clear(self) -> None: ... # wipe me before a rebuild (:69)
```

These four abstract methods are the rebuild contract, and they are worth reading as a set, because together they are the "derived, rebuildable" property made into code:

- `get_scope()` returns a `Scope` — a small declaration of which kinds of vertex this index cares about (blocks? transactions? voided ones?). The rebuild engine uses it to feed each index only the vertices relevant to it (§28.3.3, §28.5).
- `get_db_name()` returns the string under which the index records, in storage, when it was last fully built. If it returns `None`, the index keeps **no** persisted state and is therefore rebuilt on **every** startup (the in-memory mempool tips index is like this). This single return value is how the node decides, per index, whether a rebuild is needed.
- `init_loop_step(tx)` is the rebuild step: "here is one more vertex from storage; fold it into yourself." Replaying every vertex through this method reconstructs the index from scratch.
- `force_clear()` empties the index so a rebuild starts from a clean slate.

Notice what is not here: there is no `get_balance` or `get_height` on the base. Those live on the specific index interfaces (`UtxoIndex`, `HeightIndex`, ...). The base captures only what is common to all indexes — how to clear, scope, and rebuild them — so the manager can treat them uniformly.

28.3.3 `Scope`: declaring what a vertex an index wants

`Scope` (`scope.py:23`) is a tiny `NamedTuple` that makes "which vertices does this index care about?" into data rather than code:

```
# hathor/indexes/scope.py:23
class Scope(NamedTuple):
    include_blocks: bool
    include_txs: bool
    include_voided: bool
    topological_order: bool = False

    def matches(self, tx: BaseTransaction) -> bool: # :30
        if tx.get_metadata().voided_by and not self.include_voided:
            return False
        if tx.is_block and not self.include_blocks:
            return False
        if not tx.is_block and not self.include_txs:
            return False
        return True
```

Each index declares its scope as a module-level constant. Compare three real ones:

- The **UTXO** index uses `include_blocks=True, include_txs=True, include_voided=False` (`utxo_index.py:36`) — it cares about both kinds of vertex, but only executed ones, because a voided transaction's outputs are not spendable.
- The **address** index uses `include_voided=True` (`address_index.py:30`) — it indexes all history, voided included, because a wallet wants to see attempts that failed too.
- The **height** index uses `include_blocks=True, include_txs=False, include_voided=False` (`height_index.py:30`) — only executed blocks have a height; transactions and voided blocks are irrelevant to it.

The `topological_order` flag asks the rebuild iterator to deliver vertices in dependency order (parents before children) rather than arbitrary order, which some indexes need to compute their entries correctly. `matches()` and `get_iterator()` (`scope.py:30, :40`) are what the manager calls during a rebuild to route the right vertices to the right index. The payoff of expressing scope as data is in §28.5: the manager can union the scopes of all indexes that need rebuilding and make a single pass over storage feeding each one only what it asked for.

28.3.4 IndexesManager : one object owns them all

The rest of the node does not talk to fifteen separate indexes. It talks to one `IndexesManager` (`manager.py:53`), an abstract base that holds every index as a typed attribute and offers methods that fan a single event out to all of them. Its own docstring: "IndexesManager manages all the indexes that we will have in the system... so it will know which index is better to use in each moment." The attributes (`manager.py:62 - :76`) are the full roster:

```
# hathor/indexes/manager.py:62
info: InfoIndex                # global counts + first/latest timestamps
sorted_all: TimestampIndex     # every vertex, by timestamp
sorted_blocks: TimestampIndex  # blocks, by timestamp
sorted_txs: TimestampIndex     # transactions, by timestamp
height: HeightIndex            # block height → block hash
mempool_tips: MempoolTipsIndex # tips of the unconfirmed mempool
addresses: Optional[AddressIndex] # address → history (optional)
tokens: Optional[TokensIndex]   # token uid → info + supply (optional)
utxo: Optional[UtxoIndex]       # unspent outputs (optional)
nc_creation: Optional[NCCreationIndex] # nano-contracts → Ch 39 (optional)
nc_history: Optional[NCHistoryIndex] # (optional)
blueprints: Optional[BlueprintTimestampIndex] # (optional)
blueprint_history: Optional[BlueprintHistoryIndex] # (optional)
```

Two things to read off this list immediately.

Some indexes are always present; some are `Optional`. `info`, the three `sorted_*` timestamp indexes, `height`, and `mempool_tips` are non-optional — the node cannot function without them (sync needs the height tip and the mempool tips; the node needs counts and time ordering). The rest are `Optional[...]` and default to `None`; they are switched on only if a feature or flag requires them. This is the "indexes cost space and write-work" trade-off made into a runtime choice: an operator who does not run a wallet-facing API need not pay for the UTXO or address indexes at all. The `enable_address_index`, `enable_tokens_index`, `enable_utxo_index`, and `enable_nc_indexes` methods (`manager.py:108 – :126`, abstract; implemented at `:451 – :477`) build the optional index lazily, the first time it is asked for, and do nothing if it already exists.

`iter_all_indexes()` (`manager.py:90`) returns every non-`None` index as a flat iterator (`filter(None, [...])` drops the disabled ones). This is the seam that lets the manager do anything "to every index" without naming them — clear them, rebuild them, check their freshness. The first such use is a safety check, `__init_checks__` (`manager.py:78`), which asserts no two indexes claim the same `get_db_name()`, so two indexes can never overwrite each other's on-disk state.

28.3.5 Two backends, the same manager

The abstraction pattern of Chapter 27 reappears exactly. `IndexesManager` is abstract; the concrete one used in production is `RocksDBIndexesManager` (`manager.py:421`), constructed from the same `RocksDBStorage` the node already opened:

```
# hathor/indexes/manager.py:421
class RocksDBIndexesManager(IndexesManager):
    def __init__(self, rocksdb_storage, *, settings):
        self.db = rocksdb_storage.get_db()
        self.info = RocksDBInfoIndex(self.db, settings=settings)
        self.height = RocksDBHeightIndex(self.db, settings=settings)
        self.mempool_tips = MemoryMempoolTipsIndex(settings=settings) # see note below
        self.sorted_all = RocksDBTimestampIndex(self.db, scope_type=ALL, settings=settings)
        self.sorted_blocks = RocksDBTimestampIndex(self.db, scope_type=BLOCKS, settings=settings)
        self.sorted_txs = RocksDBTimestampIndex(self.db, scope_type=TXS, settings=settings)
        self.addresses = self.tokens = self.utxo = None # optional → off by default
        ...
        self.__init_checks__() # must be last
```

Two details reward attention.

The mempool tips index is in-memory even in the RocksDB manager

(`manager.py:434`). The comment explains why: "use of RocksDBMempoolTipsIndex is very slow and was suspended." So the production manager mixes backends — most indexes persist to RocksDB, but the mempool tips live in RAM and are rebuilt every boot. This is legitimate precisely because the index is derived: a RAM-only index has `get_db_name() → None` and is reconstructed from storage at startup, no different from any other rebuild. It is a clean example of the trade-off being tuned per index: the mempool is small and the rebuild is cheap, so paying the rebuild cost beats paying a slow persisted-write cost on every transaction.

The same three `TimestampIndex` objects, different scopes. `sorted_all`, `sorted_blocks`, and `sorted_txs` are the same class parameterized by a `scope_type` (`ALL` / `BLOCKS` / `TXS`). One implementation, three instances, each maintaining a time-sorted list of a different slice of the ledger. This is why the package has more index attributes than index files.

A memory-only counterpart exists for tests and the simulator (the `memory*_index.py` files), assembled the same way. The node code above the manager — the APIs, the wallet, sync — calls the same methods regardless of which backend is underneath.

28.3.6 The UTXO index — the payoff of Chapter 7

This is the index most worth walking in full, because it makes the UTXO model concrete and shows the executed/voided pattern in its clearest form. Its job: answer "which unspent outputs does this address own (optionally of this token), and what do they sum to?" — i.e. compute a spendable balance without a full scan.

The unit it stores is `UtxoIndexItem` (`utxo_index.py:38`), a frozen dataclass — one entry per indexed unspent output:

```
# hathor/indexes/utxo_index.py:38
@dataclass(frozen=True)
class UtxoIndexItem:
    token_uid: bytes      # which token this output holds (HTR = b'\x00')
    tx_id: bytes          # the transaction (or block) that created the output
    index: int            # which output of that tx
    address: str          # the address the output is locked to (the lookup key)
    amount: int           # the value, in the smallest unit
    timelock: Optional[int] # unspendable until this Unix time, if set
    heightlock: Optional[int] # unspendable until this block height (block rewards)
```

Each field models a real thing a spender needs to know. `(tx_id, index)` is the UTXO pointer of Chapter 7 — it names the exact output. `address` is the field the index is keyed on, because "find my coins" is keyed on the address. `amount` lets the index sum to a target without re-reading the output. `timelock / heightlock` are there because an output can be present-but-not-yet-spendable: a freshly-mined block reward is locked for `REWARD_SPEND_MIN_BLOCKS` blocks (`from_tx_output`, `utxo_index.py:71`), and a wallet must know that before offering the coin as change. The `from_tx_output` constructor (`:54`) builds an item from a vertex output, refusing **authority** outputs (`:57`) and outputs whose script is not a recognizable address (`:61`) — those carry no spendable, address-locked value, so they have no place in a UTXO-by-address table.

Now the heart of it: keeping the table correct as the ledger changes. The manager calls `update(tx)` (`utxo_index.py:123`) whenever a vertex's status settles, and `update` dispatches on whether the vertex is **executed** or **voided**:

```
# hathor/indexes/utxo_index.py:123
def update(self, tx: BaseTransaction) -> None:
    tx_meta = tx.get_metadata()
    if tx_meta.voided_by:
        self._update_voided(tx)      # this tx lost / is invalid → undo its effect
    else:
        self._update_executed(tx)    # this tx counts → apply its effect
```

The **executed** path (`_update_executed`, `:133`) is the index analogue of "a transaction happened": the outputs it spent are no longer unspent, and the outputs it created now are.

```
# hathor/indexes/utxo_index.py:133
def _update_executed(self, tx):
    assert not tx.get_metadata().voided_by
    for tx_input in tx.inputs:          # the outputs this tx consumed...
        spent_tx = tx.get_spent_tx(tx_input)
        spent_txout = spent_tx.outputs[tx_input.index]
        if self._should_index_output(spent_txout):
            self._remove_utxo(UtxoIndexItem.from_tx_output(spent_tx, tx_input.index, spent_txout))
    for index, tx_output in enumerate(tx.outputs): # ...the new outputs it created
        self._add_output_to_index(tx, index, tx_output)
```

Read it against the toy in §28.3.1: this is `add_record` plus a matching remove. Spending an output **removes** its item (it is no longer unspent); creating an output **adds** one. Balance queries are then just "sum the `amount` of the items for this address," with no scan of the ledger.

The **voided** path (`_update_voided`, :166) is the exact mirror, and it is why the index can be trusted across reorgs (Chapter 10). When a transaction is voided, everything it did must be undone: the outputs it created are no longer real, so they are **removed**; and the inputs it spent are un-spent again, so those prior outputs are **re-added**.

```
# hathon/indexes/utxo_index.py:166
def _update_voided(self, tx):
    for tx_input in tx.inputs:                                # outputs this tx had spent...
        spent_tx = tx.get_spent_tx(tx_input)
        spent_txout = spent_tx.outputs[tx_input.index]
        if self._should_index_output(spent_txout):
            self._add_utxo(UtxoIndexItem.from_tx_output(spent_tx, tx_input.index, spent_txout))
    for index in range(len(tx.outputs)):                      # outputs this tx created...
        self._remove_output_from_index(tx, index)             # ...disappear
```

This add/remove symmetry is the whole reason voiding can be a reversible mark (Chapter 10) rather than a deletion: when consensus flips a transaction from executed to voided or back, the index has a precise undo for every change it ever applied. Reorgs become "replay the executed/voided transitions through `update`," and the lookup table stays consistent with the canonical ledger.

One last honest note from the code: the UTXO index is documented as **not critical** and used by optional, wallet-facing APIs (`utxo_index.py:88 - :93`); it is `Optional` on the manager and `None` unless `enable_utxo_index()` is called. A node that serves no such API never builds it — the trade-off declined.

28.3.7 The other indexes, in brief

Each remaining index is the same idea — precompute a question's answer, keyed the question's way, maintain it on add/remove (executed/voided), rebuild from storage when stale — so the descriptions can be short. What differs is what each maps and who asks.

Address index (`address_index.py:37`, `AddressIndex`). Maps an **address** → **the list of transaction hashes that involve it** (as input or output), so a wallet can show "my history." Its scope includes voided vertices (`:30`), because a wallet wants to see failed attempts too — a deliberate contrast with the UTXO index, which wants only spendable (executed) outputs. It also holds an optional `pubsub` reference (`:40`) so that, in some configurations, new history for an address can be announced to subscribers (pub-sub is Chapter 30). The wallet (Ch 40) and the address-history API (Ch 36) are its main callers.

Tokens index (`tokens_index.py` , `TokensIndex`). Maps a **token uid** → **that token's metadata and running supply** — its name and symbol, how much is in circulation (minted minus melted), and which outputs/authorities exist for it. Custom tokens (Chapter 7's multi-token model) are created by `TokenCreationTransaction` , and the supply changes whenever the token is minted or melted; the index keeps a running total so "tell me about token T" and "how much of T exists?" are direct lookups instead of a scan. The manager also routes nano-contract token events here — `create_token_info_from_contract` , `add_to_total` , `destroy_token` (`manager.py:254` , `:` `266` , `:` `328`) — so contract-minted tokens show up in the same place (Chapter 39).

Height index (`height_index.py:50` , `HeightIndex`). Maps a **block height** → **the block hash at that height** on the canonical chain, and tracks the **height tip** (the highest block) via `get_height_tip` (`:` `67`). This is the chain backbone made queryable: "what is the block at height N?" and "how tall is the chain?" become O(1). It has an `add_new` path for extending the chain and an `add_reorg` path (`:` `60`) for replacing an existing height during a reorg — the executed/voided pattern in height-index clothing. Sync (Chapter 35) leans on this constantly to negotiate what each peer is missing; mining (Chapter 37) reads the tip to build the next block template.

Timestamp / sorted indexes (`timestamp_index.py` , `TimestampIndex` ; instances `sorted_all` , `sorted_blocks` , `sorted_txs`). Maintain vertices **sorted by timestamp**, one instance per slice (all / blocks-only / txs-only). They answer "give me vertices in this time window" and "walk the ledger in time order" — used by block explorers, time-range APIs, and any traversal that wants chronological order. Adding a vertex inserts it into the sorted structure (`add_tx`); voiding/removal deletes it (`del_tx`), called from the manager's `add_to_non_critical_indexes` / `del_from_non_critical_indexes` (`manager.py:348` , `:` `397`).

Mempool tips index (`mempool_tips_index.py` , `MempoolTipsIndex` ; in production the in-memory `MemoryMempoolTipsIndex`). Tracks the **tips of the mempool** — the unconfirmed transactions at the frontier of the DAG that nothing else yet confirms. From these tips the full mempool can be derived by walking down. This is the index that answers "what is pending?" and supplies parents for a newly-created transaction (a new tx must confirm two existing tips, Chapter 8). It is **critical** but **not persisted**: the manager updates it via `update_critical_indexes` (`manager.py:194`) on every vertex and via `del_from_critical_indexes` (`:` `379`) on removal, and rebuilds it from storage at every boot. Its scope makes it the one index the node updates eagerly and separately from the rest, because the mempool must always be current for mining and for accepting new transactions.

Info index (`info_index.py` , `InfoIndex`). Holds **global bookkeeping** — total vertex/transaction/block counts and the first/latest timestamps the node has seen — updated by `update_counts` / `update_timestamps` (`manager.py:346` , `:375`). It backs node-statistics and dashboard queries. Cheap to maintain (a handful of counters) and so always present.

Nano-contract indexes (`nc_creation_index.py` , `nc_history_index.py` , `blueprint_timestamp_index.py` , `blueprint_history_index.py`). Track contract creations, per-contract call history, and blueprint listings/history. They are enabled together via `enable_nc_indexes` (`manager.py:466`) only when nano-contracts are active, and the manager feeds them from contract-execution records (`non_critical_handle_contract_execution` , `:203` , with a matching unexecution path for reorgs at `:275`). They exist for the same reason as the others — "what contracts exist?", "what has this contract done?" must be cheap — and are covered with the nano-contract subsystem in **Chapter 39**.

28.3.8 Critical vs. non-critical, and why the split

You will have noticed the manager's update methods come in two flavours:

`update_critical_indexes` / `del_from_critical_indexes` (the mempool tips) and `update_non_critical_indexes` / `add_to_non_critical_indexes` / `del_from_non_critical_indexes` (everything else). The distinction is operational, not conceptual:

- **Critical** indexes (the mempool tips) must be updated immediately and in lock-step with the ledger, because the node's ability to accept and create transactions depends on them being correct right now.
- **Non-critical** indexes (UTXO, address, tokens, timestamp, info, nano) can lag slightly or be rebuilt without stopping the node, because only queries depend on them — and a query getting a momentarily-stale answer is recoverable, whereas mining on a stale mempool is not.

This split is the "derived and disposable" property turned into a performance policy: the node spends its synchronous, can't-get-it-wrong effort only on the one index that gates correctness of new transactions, and treats the rest as catch-up work.

28.4 A worked trace: one transaction moves through the indexes

Tie it together with a concrete vertex. Suppose address **A** holds one unspent 50-HTR output (created by an earlier transaction **X**, output 0), and A sends 30 HTR to address **B**, with 20 HTR change back to A. The new transaction **T** has one input — **(X, 0)** — and two outputs: 30 to B, 20 to A. Trace what each index does when **T** is **executed**:

1. **UTXO index.** `_update_executed(T)` runs (`utxo_index.py:133`): the input **(X, 0)** is removed (A's 50-HTR coin is now spent), and two items are added — 30 HTR to B, 20 HTR to A. A's spendable balance, previously `{50}`, is now `{20}`; B's gains `{30}`. No ledger scan; just three table edits.
2. **Address index.** **T**'s hash is appended to A's history (it was both spender and change-receiver) and to B's history. Both addresses can now list **T**.
3. **Timestamp indexes.** **T** is inserted into `sorted_all` and `sorted_txs` at its timestamp; a time-window query spanning that moment will now include it.
4. **Mempool tips index.** Before a block confirms it, **T** is a mempool tip — it is added so the node can offer it as a parent and report it as pending. When a later block confirms **T**, it leaves the mempool tips (`del_from_critical_indexes`).
5. **Info index.** The transaction count increments; the latest-timestamp may advance.

Now suppose a reorg later **voids T** (a heavier chain confirmed a conflicting spend of **(X, 0)**). `update(T)` now takes the voided branch (`_update_voided`, :166): the 30-to-B and 20-to-A items are removed, and the original **(X, 0)** 50-HTR item is re-added — A's spendable balance returns to `{50}`. Every other index applies its own undo (`del_from_non_critical_indexes` with `remove_all`, `manager.py:386`). The lookup tables snap back to exactly what they would have been had **T** never executed — which is the point: the indexes track the canonical ledger, and the canonical ledger just changed.

28.5 How indexes are rebuilt — and why they can be

Everything so far assumed the indexes were already correct and only needed incremental upkeep. But at boot — especially after an unclean shutdown — they may be stale or missing. Because indexes are derived, the recovery is not repair but **rebuild from storage**, and the engine for it is `_manually_initialize` (`manager.py:128`). It is the clearest demonstration in the codebase of the "derived, rebuildable" property, so it is worth reading.

The logic, step by step:

```

# hathor/indexes/manager.py:128 (condensed)
def _manually_initialize(self, tx_storage):
    db_last_started_at = tx_storage.get_last_started_at()

    indexes_to_init = []
    for index in self.iter_all_indexes():
        name = index.get_db_name()
        if name is None:
            indexes_to_init.append(index)
            continue
        if db_last_started_at != tx_storage.get_index_last_started_at(name):
            indexes_to_init.append(index)

    for index in indexes_to_init:
        index.force_clear()

    ...
    overall_scope = reduce(operator.__or__, (i.get_scope() for i in indexes_to_init))
    for tx in overall_scope.get_iterator(tx_storage):
        for index in indexes_to_init:
            if index.get_scope().matches(tx):
                index.init_loop_step(tx)

```

The mechanism rests on a **timestamp handshake**. Storage records, once, when the database was last started cleanly (`get_last_started_at`). Each persisted index records when it was last fully built (`get_index_last_started_at(name)`). If those two stamps match, the index is known-current and skipped. If they differ — or if the index keeps no persisted state at all (`get_db_name()` is `None`) — the index is **stale** and goes on the rebuild list. (Indexes that need rebuilding first have their stamp reset to `NULL_INDEX_LAST_STARTED_AT`, `manager.py:155`, so that a crash during the rebuild leaves them correctly marked stale and they are rebuilt again next time — the rebuild is itself crash-safe.)

The rebuild then does exactly what the toy in §28.3.1 promised: **clear, then replay**. `force_clear()` empties each stale index; then the manager takes the union of all their scopes (`reduce(operator.__or__, ...)`, `manager.py:170`) and makes **a single pass** over the vertices in storage, handing each vertex to every stale index whose scope `matches` it, via `init_loop_step`. One read of the ledger reconstructs every stale index at once. (Notice the care taken for performance: the storage cache is shrunk during the load and restored afterward, `manager.py:161 / :191`, because the rebuild touches every vertex once and a large cache would only waste memory.)

This is the whole argument for "indexes hold no truth" paying off operationally. The node never has to trust an index across a crash. It checks the stamp; if anything is uncertain, it clears and replays from the one thing that is trusted — the vertices in storage. An index can be wrong, deleted, or brand-new, and the node converges to a correct index the same way every time. This is also why a fresh node, or one that just enabled the UTXO index for the first time, builds that index the same way: there is no special "first build" path — building and rebuilding are one mechanism.

28.6 How it plugs into the lifecycle

Place the package in the life-of-a-node story (Chapter 0, §0.3):

```
Act I – Startup
  5. node assembled (builder)      – Ch 24 — the IndexesManager is created here, wrapping
      |                               the same RocksDB the storage layer opened
      ▼
  6. manager starts → INITIALIZATION
      |
      | _manually_initialize(): check each index's stamp;
      | clear + replay storage into any stale index  ◀ THIS CHAPTER (§28.5)
      | (this is "indexes are prepared", Ch 0 §0.3 step 6)
      ▼
  7. reactor runs → STEADY STATE
      ingestion (Ch 33) executes/voids a vertex →
      consensus (Ch 32) sets voided_by / first_block / spent_outputs →
      manager.update_critical_indexes()      (mempool tips, eager)
      manager.update_non_critical_indexes()  (UTXO, address, tokens, ...)
      APIs / wallet / sync / mining READ the indexes – Ch 36, 40, 35, 37
```

The sequence: the **builder** (Chapter 24) constructs the `IndexesManager` over the already-open RocksDB and hands it to the `HathorManager` (Chapter 29). During the manager's **initialization**, `_manually_initialize` runs — this is the "indexes are prepared" line in the orientation chapter's boot narrative — bringing every index into agreement with storage before the node serves anyone. In **steady state**, the **ingestion pipeline** (Chapter 33) and **consensus** (Chapter 32) are the writers: when a vertex is executed or voided, they update the vertex's metadata and then call the manager's update methods to keep the indexes in step. Everyone else is a reader: the wallet (Chapter 40) reads the address and UTXO indexes for balances and history; the HTTP/WebSocket APIs (Chapter 36) read all of them; sync (Chapter 35) reads the height index and mempool tips; mining (Chapter 37) reads the height tip and mempool tips to build templates.

The clean division to carry forward: **storage is written first and is the source of truth; the indexes are written second and are derived from it**. A reader who wants the fact about a vertex goes to storage by hash; a reader who wants to find vertices by any other key goes to an index. And if the indexes and storage ever disagree, storage wins, because the indexes can always be rebuilt from it.

Recap

Index	Maps (key → value)	Who asks
UTXO (<code>utxo_index.py</code>)	(address, token) → unspent <code>UtxoIndexItems</code>	wallets / spend-building APIs — "what can I spend?"

Index	Maps (key → value)	Who asks
Address (<code>address_index.py</code>)	address → its transaction hashes (incl. voided)	wallet, address-history API — "my history?"
Tokens (<code>tokens_index.py</code>)	token uid → info + running supply	token info/supply APIs (incl. nano-contract tokens)
Height (<code>height_index.py</code>)	block height → block hash; the height tip	sync, mining, explorers — "the chain backbone"
Timestamp (<code>timestamp_index.py</code>)	timestamp → vertices (all / blocks / txs)	time-window queries, ordered traversal
Mempool tips (<code>mempool_tips_index.py</code>)	the unconfirmed DAG frontier	mining + new-tx parents — "what's pending?" (critical, RAM-only)
Info (<code>info_index.py</code>)	global counts + first/latest timestamps	node statistics / dashboards
Nano-contract (<code>nc_*</code> , <code>blueprint_*</code>)	contract/blueprint creations + histories	Chapter 39 (enabled with nano-contracts)

Indexes are the node's derived lookup tables: precomputed answers to every question that is not "give me the vertex with this hash." Each spends disk space and per-write effort to turn a full-ledger scan into a direct lookup, keyed the way the question is asked — by address, token, height, time, or mempool membership. The `IndexesManager` owns them all behind one interface, offered in interchangeable memory and RocksDB backends, with optional indexes switched on only when a feature needs them. The defining property is that an index holds **no truth of its own**: every entry is recomputable from the vertices in storage, which is why the node updates them with a precise executed/voided symmetry, why reorgs are just replayed transitions, and why boot recovery is a blunt clear-and-replay rather than delicate repair. Storage is the source of truth; the indexes are how the node finds things in it. The next chapter steps up to the object that owns both — and the whole node besides: **Chapter 29, the `HathorManager`**, the coordinator that brings storage, indexes, and every other subsystem to life and drives them through the lifecycle this part of the book has been tracing.

-
1. An **index**, in databases, is a derived (secondary) data structure that precomputes the answer to a class of lookup queries, keyed the way the query is phrased, so the answer can be found without scanning all the primary data. It costs extra storage and extra work on every write, in exchange for

fast reads. The index at the back of a book — topic → page numbers — is the same idea on paper. Because it is computed from the primary data, it can always be rebuilt and contains no information that is not already implied by that data. ↵

2. Derived state (also secondary state) is data computed from some primary source of truth, holding nothing the primary does not already imply. Its defining property is that it can be discarded and recomputed. Hathor's indexes are derived from the vertices in storage; storage is primary. This is what makes index corruption recoverable by rebuild rather than fatal. ↵
3. A **tip** of the ledger DAG is a vertex that no other vertex confirms yet — a leaf at the growing frontier of the graph. A **mempool tip** is such a tip that is also still unconfirmed by any block. New transactions attach to (confirm) existing tips, so the node must always know what its current tips are. Full treatment of the DAG and tips in Chapter 8. ↵